

Empirical Textual Diversity and Predictive Risk in Character-Level Autoregressive Models

Nicolás Meléndez
Universidad de los Andes, Bogotá
nicolasmelendez30@gmail.com

2026

Abstract

We study whether corpora with nearly equal character budgets but different empirical diversity induce systematic differences in the predictive risk of small autoregressive models. The statistical objects implemented by the software are specified explicitly: a finite sample space of strings, empirical measures on overlapping context windows, conditional probability kernels defined by causal decoders, and a finite-sample optimization problem. The experiment consists of one Transformer and one RMSNorm–RoPE–SwiGLU variant, each trained once on a homogeneous corpus and on a mixture of authors. The mixture has larger block entropy, distinct-4 ratio, and gzip byte ratio. Its test loss is nevertheless larger for both architectures, whereas its empirical test–train difference is smaller. These four fits do not identify a causal effect of diversity: character distribution, domain, vocabulary, and parameter count vary simultaneously, and only one initialization was observed. We therefore treat the result as a computational description rather than a population estimate. A unigram-adjusted contextual gain and a replicated factorial design are specified for subsequent experiments.

1 Problem statement

Let $\mathcal{A}$ be a finite alphabet and let $s = (s_0, \dots, s_{N-1}) \in \mathcal{A}^N$ be an observed string. No independence or stationarity assumption is imposed on its coordinates. Empirical scaling laws relate loss to parameter count, data, and computation [1, 2]; they do not imply that two strings of the same length define statistically equivalent learning problems. Chang et al. introduce quality corrections that replace raw token count by an effective size [3]. The present experiment is not a replication of that fit: it estimates no scaling coefficient, constructs no scaling curve, and uses no teacher model.

For two strings $s^{(0)}$ and $s^{(1)}$ of approximately equal length, two neural families, and a fixed optimization budget, we compare

$$L_j = \hat{R}_{\text{te}}(\hat{w}; s^{(j)}), \quad (1)$$

$$\Gamma_j = \hat{R}_{\text{te}}(\hat{w}; s^{(j)}) - \hat{R}_{\text{tr}}(\hat{w}; s^{(j)}), \quad j \in \{0, 1\}. \quad (2)$$

L_j is absolute predictive loss on a held-out segment of the same corpus. Γ_j compares two empirical measures supported on different windows. It is not itself a population generalization bound. Neither quantity identifies a causal effect of diversity without further design assumptions.

2 Measure-theoretic foundations

This section fixes the probabilistic language used below. The material is standard; proofs are included when they clarify a construction used by the software. General versions can be found in [4].

2.1 Measurable spaces and probability measures

Definition 2.1 (σ -algebra). *Let Ω be a set. A collection $\mathcal{F} \subseteq 2^\Omega$ is a σ -algebra if $\Omega \in \mathcal{F}$, if it is closed under complements, and if it is closed under countable unions. The pair $(\Omega, \mathcal{F})$ is a measurable space.*

On a finite set S we use the power set 2^S . On $\mathbb{R}^d$ we use the Borel σ -algebra $\mathcal{B}(\mathbb{R}^d)$ generated by the Euclidean open sets.

Definition 2.2 (Measure). *A measure on $(\Omega, \mathcal{F})$ is a function $\mu : \mathcal{F} \rightarrow [0, \infty]$ such that $\mu(\emptyset) = 0$ and*

$$\mu\left(\bigcup_{n=1}^{\infty} A_n\right) = \sum_{n=1}^{\infty} \mu(A_n)$$

for every sequence of pairwise disjoint $A_n \in \mathcal{F}$. It is a probability measure if $\mu(\Omega) = 1$. A probability space is a triple $(\Omega, \mathcal{F}, \mathbb{P})$.

For $\omega \in \Omega$, the Dirac measure is $\delta_\omega(A) = \mathbf{1}\{\omega \in A\}$. Hence the empirical measure of $\omega_1, \dots, \omega_n$ is $n^{-1} \sum_i \delta_{\omega_i}$. This formula, rather than informal language about a “dataset distribution”, defines every empirical risk in this paper.

Definition 2.3 (Random variable and law). *Let $(\Omega, \mathcal{F})$ and $(S, \mathcal{S})$ be measurable spaces. A map $X : \Omega \rightarrow S$ is measurable if $X^{-1}(A) \in \mathcal{F}$ for every $A \in \mathcal{S}$. If X is defined on a probability space, it is an S -valued random variable. Its law is the pushforward measure*

$$\mathbb{P}_X = \mathbb{P} \circ X^{-1}, \quad \mathbb{P}_X(A) = \mathbb{P}(X \in A).$$

If S is finite, a probability law is a vector $(p_s)_{s \in S}$ with $p_s \geq 0$ and $\sum_s p_s = 1$. Integration then reduces to summation:

$$\mathbb{E}[g(X)] = \int_{\Omega} g(X(\omega)) d\mathbb{P}(\omega) = \sum_{s \in S} g(s) p_s.$$

2.2 Conditioning

Definition 2.4 (Conditional expectation). *Let Z be integrable and $\mathcal{G} \subseteq \mathcal{F}$. A conditional expectation $\mathbb{E}[Z \mid \mathcal{G}]$ is a $\mathcal{G}$ -measurable integrable random variable satisfying*

$$\int_G \mathbb{E}[Z \mid \mathcal{G}] d\mathbb{P} = \int_G Z d\mathbb{P} \quad \text{for every } G \in \mathcal{G}.$$

It is unique up to almost-sure equality.

If X, Y are finite-valued with joint mass $p(x, y)$ and $p_X(x) > 0$, then

$$p(y \mid x) = \frac{p(x, y)}{p_X(x)}, \quad \mathbb{E}[g(Y) \mid X = x] = \sum_y g(y) p(y \mid x).$$

The conditional mass $p(\cdot \mid x)$ is the object approximated by a next-token model.

Proposition 2.5 (Tower property; standard). *If Z is integrable and $\mathcal{H} \subseteq \mathcal{G} \subseteq \mathcal{F}$, then*

$$\mathbb{E}[\mathbb{E}[Z \mid \mathcal{G}] \mid \mathcal{H}] = \mathbb{E}[Z \mid \mathcal{H}] \quad \text{almost surely.}$$

In particular, $\mathbb{E}[\mathbb{E}[Z \mid \mathcal{G}]] = \mathbb{E}[Z]$.

2.3 Stochastic processes and probability kernels

Definition 2.6 (Stochastic process and filtration). *For an index set T and a measurable state space S , a stochastic process is a family $(X_t)_{t \in T}$ of S -valued random variables on one probability space. For $T = \mathbb{N}_0$, its natural filtration is*

$$\mathcal{F}_t^X = \sigma(X_0, \dots, X_t),$$

the information generated up to time t .

Definition 2.7 (Markov property). *A discrete process (X_t) is a first-order Markov process if, for every t and every state a ,*

$$\mathbb{P}(X_{t+1} = a \mid \mathcal{F}_t^X) = \mathbb{P}(X_{t+1} = a \mid X_t) \quad \text{almost surely.}$$

An autoregressive language model need not be first-order Markov: its kernel may depend on the whole available context. The bigram model is first-order Markov. A causal decoder instead satisfies an adaptedness requirement: its logits at time t may depend on $\mathcal{F}_t^X$ but not on future tokens.

Definition 2.8 (Probability kernel). *For measurable spaces $(S, \mathcal{S})$ and $(T, \mathcal{T})$, a probability kernel K from S to T satisfies: (i) $K(s, \cdot)$ is a probability measure for each s ; and (ii) $s \mapsto K(s, A)$ is measurable for every $A \in \mathcal{T}$.*

On finite alphabets measurability is automatic. The chain rule for a joint mass function is a product of successive kernels; this is the measure-theoretic content of autoregressive factorization.

3 Statistical learning from first principles

Let $(\mathcal{U}, \mathcal{U}_0)$ be an input space, $(\mathcal{Y}, \mathcal{Y}_0)$ an output space, and $\mathcal{O}$ an action space.

Definition 3.1 (Hypothesis, loss, and risk). *A hypothesis is a measurable map $h : \mathcal{U} \rightarrow \mathcal{O}$. A hypothesis class $\mathcal{H}$ is a set of hypotheses. Given a measurable loss $\ell : \mathcal{O} \times \mathcal{Y} \rightarrow [0, \infty]$ and a data law P on $\mathcal{U} \times \mathcal{Y}$, the population risk is*

$$R_P(h) = \int \ell(h(u), y) dP(u, y).$$

Definition 3.2 (Bayes risk and approximation error). *The Bayes and class-restricted risks are*

$$R_P^* = \inf_{h \text{ measurable}} R_P(h), \quad R_{P, \mathcal{H}}^* = \inf_{h \in \mathcal{H}} R_P(h).$$

The quantity $R_{P, \mathcal{H}}^ - R_P^* \geq 0$ is the approximation error of $\mathcal{H}$ under P and ℓ .*

For observations $Z_i = (U_i, Y_i)$, define

$$\hat{P}_n = \frac{1}{n} \sum_{i=1}^n \delta_{Z_i}, \quad \hat{R}_n(h) = \int \ell(h(u), y) d\hat{P}_n = \frac{1}{n} \sum_{i=1}^n \ell(h(U_i), Y_i).$$

Definition 3.3 (Learning algorithm). *A learning algorithm is a measurable map $\mathfrak{A} : (\mathcal{U} \times \mathcal{Y})^n \rightarrow \mathcal{H}$. Its output on $S = (Z_1, \dots, Z_n)$ is $\hat{h}_S = \mathfrak{A}(S)$. An empirical risk minimizer belongs to $\operatorname{argmin}_{h \in \mathcal{H}} \hat{R}_n(h)$, when this set is nonempty.*

The trained network is not known to be an exact empirical risk minimizer. It is the output of a finite randomized algorithm; parameter initialization and minibatch indices are part of that algorithm's randomness.

Proposition 3.4 (Excess-risk decomposition; algebraic). *Suppose $h_{\mathcal{H}}^*$ minimizes R_P over $\mathcal{H}$. For every $\hat{h} \in \mathcal{H}$,*

$$\begin{aligned} R_P(\hat{h}) - R_P^* &= \underbrace{R_{P,\mathcal{H}}^* - R_P^*}_{\text{approximation}} \\ &\quad + \underbrace{R_P(\hat{h}) - \hat{R}_n(\hat{h}) + \hat{R}_n(h_{\mathcal{H}}^*) - R_P(h_{\mathcal{H}}^*)}_{\text{estimation/generalization}} \\ &\quad + \underbrace{\hat{R}_n(\hat{h}) - \hat{R}_n(h_{\mathcal{H}}^*)}_{\text{empirical optimization}}. \end{aligned}$$

Proof. Add and subtract $\hat{R}_n(\hat{h})$, $\hat{R}_n(h_{\mathcal{H}}^*)$, and $R_P(h_{\mathcal{H}}^*)$; then group terms. The identity uses no probabilistic assumption. $\square$

The decomposition explains why a cross-corpus loss difference cannot be assigned to one mechanism. Changing a corpus may alter Bayes risk, approximation error, finite-sample deviation, and the optimization landscape.

If $Z_1, \dots, Z_n$ are iid from P and h is fixed independently of the sample, then $\mathbb{E}[\hat{R}_n(h)] = R_P(h)$. If also $0 \leq \ell \leq M$, Hoeffding's inequality yields

$$\mathbb{P}(|\hat{R}_n(h) - R_P(h)| \geq \varepsilon) \leq 2 \exp(-2n\varepsilon^2/M^2).$$

For a data-dependent $\hat{h}$, a pointwise bound is insufficient. One needs uniform complexity control, stability, PAC-Bayes analysis, or another learning theorem. None applies automatically to overlapping text windows, so no population generalization bound is claimed here.

4 Neural networks as parametric maps

4.1 Classical feedforward networks

Definition 4.1 (Affine layer). *For $m, n \in \mathbb{N}$, an affine layer is $A_{W,b} : \mathbb{R}^n \rightarrow \mathbb{R}^m$, $A_{W,b}(x) = Wx + b$, where $W \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$. It has $mn + m$ scalar parameters.*

An activation is a measurable scalar map extended coordinatewise. The models use

$$\text{GELU}(x) = x\Phi(x), \quad \text{SiLU}(x) = \frac{x}{1 + e^{-x}},$$

where Φ is the standard normal distribution function [5, 6].

Definition 4.2 (Feedforward neural network). *Let $d_0, \dots, d_L \in \mathbb{N}$. A depth- L feedforward network with activation ρ is the parametrized family $\Phi_\theta : \mathbb{R}^{d_0} \rightarrow \mathbb{R}^{d_L}$ given by*

$$\begin{aligned} h_0 &= x, \\ h_\ell &= \rho(W_\ell h_{\ell-1} + b_\ell), & 1 \leq \ell < L, \\ \Phi_\theta(x) &= W_L h_{L-1} + b_L, \end{aligned}$$

where θ concatenates all matrices and biases. Its hypothesis class is $\{\Phi_\theta : \theta \in \Theta\}$.

Modern architectures extend this base definition by composing differentiable modules in a directed acyclic computation graph. Embedding lookup, normalization, residual addition, attention, and multiplicative gating are such modules. The Transformer is a neural network in this compositional sense.

4.2 Embeddings and tensor-valued layers

An embedding table is $E \in \mathbb{R}^{K \times d}$ with lookup $i \mapsto E_{i,:}$. Equivalently, $E_{i,:} = e_i^\top E$ for a one-hot vector $e_i \in \mathbb{R}^K$. Embedding lookup is therefore linear after the canonical inclusion of a token into Euclidean space.

A sequence model acts on $H \in \mathbb{R}^{B \times d}$. Positionwise linear maps have the form $H \mapsto HW$; normalization and activations act rowwise; attention couples rows. Every finite tensor can be vectorized, so the architecture remains a finite-dimensional parametrized map.

4.3 Differentiation and backpropagation

Proposition 4.3 (Chain rule for a computation graph; standard). *Let $F = f_L \circ \dots \circ f_1$, with differentiable $f_j : \mathbb{R}^{d_{j-1}} \rightarrow \mathbb{R}^{d_j}$. If $h_j = f_j(h_{j-1})$, then*

$$DF(x) = Df_L(h_{L-1}) \cdots Df_1(x).$$

For a scalar objective, reverse-mode differentiation evaluates the associated transpose-Jacobian products from L to 1 and returns the gradient, up to floating-point arithmetic.

Proof. The formula is the multivariate chain rule applied inductively. Associativity permits the Jacobian product to be evaluated in reverse order without forming each full Jacobian. $\square$

Backpropagation is thus an efficient chain-rule algorithm, not an optimizer. The resulting gradient is passed to SGD, AdamW, or another optimizer [7].

Three questions must remain separate. *Representation* asks which maps a fixed architecture realizes exactly. *Approximation* asks how closely a growing architecture family approaches a target. *Optimization* asks whether an algorithm reaches a good parameter. Universal approximation, when available, answers only the second question; it implies neither learnability from finite data nor convergence of AdamW. No universal-approximation claim is used in this experiment.

5 Discrete representation and sample construction

5.1 Character tokenization

Definition 5.1 (Corpus-dependent tokenizer). *For a string s , define*

$$\mathcal{V}(s) = \{a \in \mathcal{A} : \text{there exists } i \text{ such that } s_i = a\}.$$

A character tokenizer is a bijection

$$\tau_s : \mathcal{V}(s) \longrightarrow \{0, \dots, |\mathcal{V}(s)| - 1\},$$

extended coordinatewise to $\mathcal{V}(s)^n$ for every $n \in \mathbb{N}$. The implementation orders $\mathcal{V}(s)$ lexicographically and stores both τ_s and τ_s^{-1} .

The transformation preserves length exactly. Since $\mathcal{V}(s)$ is constructed from the complete string, including its test segment, no out-of-vocabulary event can occur. This is a limited transductive use of test information: token frequencies and order remain hidden, but the support is known before fitting. A stricter protocol would construct a common vocabulary from training data and include an unknown symbol.

Corpus-specific vocabularies also alter model capacity. If $|\mathcal{V}(s^{(0)})| \neq |\mathcal{V}(s^{(1)})|$, the input embedding and output projection have different dimensions. Losses are therefore reported in nats per character, and parameter counts must be reported alongside them.

5.2 Overlapping windows

Fix a context length $B \in \mathbb{N}$. For $x_i = \tau_s(s_i)$ define

$$W_B(s)_i = (X_i, Y_i), \quad 0 \leq i < N - B, \quad (3)$$

$$X_i = (x_i, \dots, x_{i+B-1}), \quad (4)$$

$$Y_i = (x_{i+1}, \dots, x_{i+B}). \quad (5)$$

The string is first divided into contiguous train, validation, and test intervals with proportions 0.8, 0.1, and 0.1. Windows are then formed inside each interval, so no example crosses a split boundary.

Proposition 5.2 (Deterministic overlap; project-specific). *For every admissible i , the input windows X_i and X_{i+1} share exactly $B - 1$ token positions. Consequently, the collection $\{W_B(s)_i\}_{i=0}^{N-B-1}$ cannot be an iid sample unless the word “sample” refers only to randomized index selection from the finite collection.*

Proof. The suffix $(x_{i+1}, \dots, x_{i+B-1})$ of X_i is the prefix of X_{i+1} of the same length. The equality is deterministic and is independent of any probabilistic law assigned to the original string. $\square$

The reported standard deviations across randomly selected evaluation batches therefore describe Monte Carlo variation of the implemented estimator. Without a dependence model—for example, stationarity plus a suitable mixing condition—the usual iid standard-error interpretation is unavailable.

6 Probability model

Let $K = |\mathcal{V}(s)|$ and $\Delta_{K-1} = \{p \in [0, 1]^K : \sum_{j=0}^{K-1} p_j = 1\}$. For a fixed architecture, $w \in \Theta \subset \mathbb{R}^p$ parametrizes a map

$$f_w : \{0, \dots, K-1\}^{\leq B} \longrightarrow \mathbb{R}^{B \times K}.$$

The domain carries its power σ -algebra and the codomain its Borel σ -algebra. Every such map is measurable because the domain is finite.

Definition 6.1 (Autoregressive probability kernel). *For a context $x_{\leq t}$, define*

$$Q_w(j \mid x_{\leq t}) = \text{softmax}(f_w(x_{\leq t})_{t,:})_j, \quad (6)$$

$$\text{softmax}(z)_j = \frac{e^{z_j}}{\sum_{r=0}^{K-1} e^{z_r}}. \quad (7)$$

For fixed $x_{\leq t}$, $Q_w(\cdot \mid x_{\leq t}) \in \Delta_{K-1}$.

Given an initial distribution μ on $\mathcal{V}$, the finite-horizon law is

$$Q_w^{(T)}(x_0, \dots, x_{T-1}) = \mu(x_0) \prod_{t=0}^{T-2} Q_w(x_{t+1} \mid x_{\leq t}). \quad (8)$$

Because the state space is finite, consistency of these kernels gives an infinite-sequence law by the Ionescu–Tulcea construction. The implementation only requires the finite-dimensional distributions in (8).

Generation temperature $T_0 > 0$ replaces z by z/T_0 . Top- k filtering restricts the support to the indices of the k largest logits and renormalizes the resulting mass. Both operations modify the generative kernel; neither appears in the training objective.

7 Loss and risk

7.1 Logarithmic score

For $z \in \mathbb{R}^K$ and $y \in \{0, \dots, K-1\}$, set

$$\ell(z, y) = -\log \text{softmax}(z)_y, \quad L_B(w; X, Y) = \frac{1}{B} \sum_{t=0}^{B-1} \ell(f_w(X)_{t,:}, Y_t).$$

The derivative with respect to z is

$$\nabla_z \ell(z, y) = \text{softmax}(z) - e_y.$$

Thus backpropagation begins with the difference between the predicted probability vector and the one-hot observation.

Proposition 7.1 (Conditional cross-entropy decomposition; standard). *Let P be a probability measure on $\mathcal{C} \times \mathcal{V}$, where $\mathcal{C}$ is a finite context space, and let $q(\cdot | c)$ be strictly positive. Then*

$$\mathbb{E}_P[-\log q(Y | C)] = H_P(Y | C) + \mathbb{E}_{P_C}[D_{\text{KL}}(P(\cdot | C) \| q(\cdot | C))].$$

Proof. Condition on $C = c$, add and subtract $\log P(Y | c)$, and sum over c with respect to P_C . The first term is conditional entropy and the second is the conditional Kullback–Leibler divergence. $\square$

This identity is essential for interpreting cross-corpus loss. A larger test loss can reflect larger irreducible conditional entropy, larger approximation error, larger optimization error, or any combination of them. Raw loss does not isolate model quality from task difficulty.

7.2 Population and empirical measures

Let $\mathbb{P}$ be a probability measure on $\mathcal{X} = \mathcal{V}^B \times \mathcal{V}^B$. Its population risk is

$$R_{\mathbb{P}}(w) = \int_{\mathcal{X}} L_B(w; x, y) d\mathbb{P}(x, y).$$

For a finite set of valid indices I , define the window empirical measure

$$\hat{\mathbb{P}}_I = \frac{1}{|I|} \sum_{i \in I} \delta_{W_B(s)_i}, \quad \hat{R}_I(w) = \int L_B(w; x, y) d\hat{\mathbb{P}}_I(x, y).$$

This definition is valid without an iid assumption. Such an assumption becomes necessary only when $\hat{R}_I$ is used to infer properties of an external population law.

7.3 Unigram adjustment

To separate part of the marginal difficulty, the current evaluator fits on the training split the add- α unigram model

$$\hat{q}_\alpha(a) = \frac{n_{\text{tr}}(a) + \alpha}{N_{\text{tr}} + \alpha K}, \quad \alpha = 1,$$

and computes

$$L_{\text{uni}} = -\frac{1}{N_{\text{te}}} \sum_{t \in I_{\text{te}}} \log \hat{q}_\alpha(x_t), \tag{9}$$

$$G_{\text{ctx}} = L_{\text{uni}} - \hat{R}_{\text{te}}(\hat{w}), \tag{10}$$

$$G_{\text{rel}} = G_{\text{ctx}} / L_{\text{uni}}. \tag{11}$$

G_{ctx} measures predictive gain over corpus-specific marginal frequencies. It does not remove domain shift or identify a causal effect. Bits per character and perplexity are

$$\text{bpc} = \hat{R}_{\text{te}} / \log 2, \quad \text{PPL} = \exp(\hat{R}_{\text{te}}).$$

They are monotone transformations of the same loss and must not be treated as independent evidence.

8 Model classes

8.1 Bigram kernel

The reference model has $A \in \mathbb{R}^{K \times K}$ and

$$Q_A(x_{t+1} = j \mid x_{\leq t}) = \text{softmax}(A_{x_t, :})_j.$$

It is a homogeneous first-order Markov kernel. Its implementation as an embedding table is exactly this parameterization, not an approximation to it.

Proposition 8.1 (Closed-form bigram maximum likelihood; standard). *Let n_{ij} be the number of observed transitions $i \rightarrow j$ and $n_i = \sum_j n_{ij}$. Among all row-stochastic matrices P , every maximum of*

$$\sum_{i,j} n_{ij} \log P_{ij}$$

satisfies $P_{ij} = n_{ij}/n_i$ whenever $n_i > 0$. Rows with $n_i = 0$ are not identified by the likelihood.

Proof. The objective separates by rows. For a fixed i with $n_i > 0$, divide by n_i and write $\hat{p}_{ij} = n_{ij}/n_i$. Then

$$-\sum_j \hat{p}_{ij} \log P_{ij} = H(\hat{p}_i) + D_{\text{KL}}(\hat{p}_i \| P_i),$$

which is minimized exactly at $P_i = \hat{p}_i$ on the observed support. $\square$

The trainable logit matrix is redundant: adding a constant to every entry of a row leaves its softmax unchanged. This non-identifiability is benign for the induced transition kernel but means the parameter minimizer is never unique without an additional convention or penalty.

8.2 Causal Transformer

Each symbol is represented by $E(x_t) + P_t \in \mathbb{R}^d$. A pre-normalized block is

$$\begin{aligned} H' &= H + \text{MHA}(\text{LN}(H)), \\ H^+ &= H' + \text{FFN}(\text{LN}(H')). \end{aligned}$$

For one head, set $Q = HW_Q$, $K' = HW_K$, and $V = HW_V$. Then

$$\text{Attn}(H) = \text{softmax}_{\text{row}} \left(\frac{QK'^{\top}}{\sqrt{d_h}} + M \right) V, \quad (12)$$

$$M_{ts} = \begin{cases} 0, & s \leq t, \\ -\infty, & s > t. \end{cases} \quad (13)$$

The output at position t is therefore a function only of $x_{\leq t}$. The repository tests this property by perturbing future tokens and checking exact agreement of earlier logits. The feedforward sublayer is $W_2 \text{GELU}(W_1 h + b_1) + b_2$; a final affine map produces K logits [8].

For finite logits, each unmasked attention row is a probability vector. Hence the pre-projection head output at position t is a convex combination of $v_0, \dots, v_t$. This does not make the complete block a convex map: queries, keys, values, softmax weights, residual paths, and the feedforward network all depend nonlinearly on the input and parameters.

Proposition 8.2 (Causal invariance; architecture-specific). *Let F_w be a stack of the implemented masked attention blocks with positionwise normalization, positionwise feedforward maps, and residual connections. If two token sequences x, x' agree through position t , then their logits agree through position t :*

$$x_{0:t} = x'_{0:t} \implies F_w(x)_{0:t,:} = F_w(x')_{0:t,:}.$$

Proof. Token and positional embeddings agree through t . Assume the hidden states entering a block agree through t . Positionwise maps preserve this agreement. At a position $r \leq t$, the mask restricts attention to indices $s \leq r$, on which the hidden states agree; therefore queries, keys, values, weights, and attention outputs agree. Residual addition preserves equality. Induction over the two sublayers and then over blocks proves the claim; the final rowwise normalization and output projection preserve it. $\square$

8.3 RMSNorm–RoPE–SwiGLU variant

The second model preserves the residual causal decoder and changes three local operators [9]. First,

$$\text{RMSNorm}(h) = g \odot \frac{h}{\sqrt{d^{-1}\|h\|_2^2 + \epsilon}}.$$

Second, RoPE applies the planar rotation $R(t\theta_i)$ to each coordinate pair, where $\theta_i = 10000^{-2i/d_h}$ [10]. Since

$$R(t\theta)^\top R(s\theta) = R((s-t)\theta),$$

the inner product between a rotated query and key depends on their relative position. Third,

$$\text{SwiGLU}(h) = W_d(\text{SiLU}(W_g h) \odot W_u h).$$

The term “LLaMA-style” refers only to these architectural choices; it does not claim equivalence in scale, training regime, tokenizer, or every implementation detail.

9 Optimization theory

9.1 First-order geometry

Let $F : \mathbb{R}^p \rightarrow \mathbb{R}$ be differentiable. The gradient is characterized by

$$F(w+h) = F(w) + \langle \nabla F(w), h \rangle + o(\|h\|).$$

Under the Euclidean norm, the unit direction of largest first-order decrease is the normalized negative gradient when $\nabla F(w) \neq 0$:

$$d_*(w) = -\frac{\nabla F(w)}{\|\nabla F(w)\|}.$$

Gradient descent therefore uses $w_{k+1} = w_k - \eta_k \nabla F(w_k)$.

Definition 9.1 (Smoothness and convexity). *A differentiable F is L -smooth if*

$$\|\nabla F(u) - \nabla F(v)\| \leq L\|u - v\| \quad \text{for all } u, v.$$

It is convex if $F(v) \geq F(u) + \langle \nabla F(u), v - u \rangle$ for all u, v , and μ -strongly convex if the right-hand side can be augmented by $\frac{\mu}{2}\|v - u\|^2$.

Lemma 9.2 (Descent lemma; standard). *If F is L -smooth, then*

$$F(v) \leq F(u) + \langle \nabla F(u), v - u \rangle + \frac{L}{2}\|v - u\|^2.$$

Consequently, for $0 < \eta < 2/L$,

$$F(u - \eta \nabla F(u)) \leq F(u) - \eta(1 - L\eta/2)\|\nabla F(u)\|^2.$$

Proof. Apply the fundamental theorem of calculus to $t \mapsto F(u + t(v - u))$, subtract the linear term at u , and use the Lipschitz bound on the gradient. Substitute $v = u - \eta \nabla F(u)$ for the second assertion. $\square$

Proposition 9.3 (Linear convergence in the ideal convex regime; standard). *If F is L -smooth and μ -strongly convex with minimizer w^* , then gradient descent with $\eta = 1/L$ satisfies*

$$F(w_k) - F(w^*) \leq \left(1 - \frac{\mu}{L}\right)^k (F(w_0) - F(w^*)).$$

This is a reference theorem, not a theorem about the experiment. Neural-network objectives are nonconvex and need not be globally smooth with a useful common constant. The statement isolates the roles of step size, curvature, and the condition number in the regime where a complete theory is available.

9.2 Stochastic gradients

Suppose $F(w) = n^{-1} \sum_{i=1}^n f_i(w)$. If I is uniform on $\{1, \dots, n\}$, then $G(w, I) = \nabla f_I(w)$ satisfies $\mathbb{E}[G(w, I)] = \nabla F(w)$. A minibatch average sampled with replacement remains unbiased and has covariance reduced by the batch size. This is randomness over a fixed finite objective; it does not make overlapping windows iid draws from a text-generating population.

Classical stochastic approximation requires assumptions such as unbiasedness, controlled second moments, appropriate step sizes, and convexity or weaker regularity. Adam introduces coordinatewise adaptive preconditioning; results for plain SGD do not transfer automatically [11].

9.3 The implemented AdamW update

Let $J(w) = \widehat{R}_{I_{\text{tr}}}(w)$. At step k , a minibatch of indices B_k is sampled uniformly with replacement and

$$g_k = \frac{1}{|B_k|} \sum_{i \in B_k} \nabla_w L_B(w_k; W_B(s)_i).$$

Conditionally on w_k , g_k is unbiased for the gradient of the finite-window objective. Overlap between the underlying windows does not change this finite-population statement; it does affect population interpretations.

AdamW performs [11, 12]

$$\begin{aligned} m_k &= \beta_1 m_{k-1} + (1 - \beta_1) g_k, \\ v_k &= \beta_2 v_{k-1} + (1 - \beta_2) g_k^{\odot 2}, \\ \hat{m}_k &= m_k / (1 - \beta_1^k), & \hat{v}_k &= v_k / (1 - \beta_2^k), \\ w_{k+1} &= (1 - \eta\lambda) w_k - \eta \hat{m}_k \oslash (\sqrt{\hat{v}_k} + \varepsilon). \end{aligned}$$

The multiplicative factor $(1 - \eta\lambda)$ distinguishes decoupled weight decay from adding an ℓ^2 penalty to an adaptive gradient. No global convergence statement follows for the nonconvex neural objectives considered here.

If S steps use minibatch size M and context length B , the number of target positions processed is SMB . Here it is $5000 \cdot 32 \cdot 128 = 20,480,000$ per fit. This count includes repeated windows and is an optimization budget, not a count of distinct observations.

10 Empirical diversity functionals

For $1 \leq n \leq N$ and $a \in \mathcal{A}^n$, define the empirical block law

$$\hat{p}_{n,s}(a) = \frac{1}{N - n + 1} \sum_{i=0}^{N-n} \mathbf{1}\{(s_i, \dots, s_{i+n-1}) = a\}.$$

The implemented functionals are

$$H_n(s) = - \sum_{a \in \mathcal{A}^n} \hat{p}_{n,s}(a) \log \hat{p}_{n,s}(a), \quad (14)$$

$$D_n(s) = \frac{|\{a : \hat{p}_{n,s}(a) > 0\}|}{N - n + 1}. \quad (15)$$

H_n is block entropy, not entropy rate. Under stationarity, the increments $H_n - H_{n-1}$ are finite-order conditional entropies and form a nonincreasing sequence converging to the entropy rate. The plug-in estimates used here do not inherit that interpretation without assumptions and finite-sample analysis. Moreover, D_n depends strongly on both N and n ; it is not an intrinsic property unless those quantities are fixed.

For the UTF-8 byte string $b(s)$, define

$$\rho_{\text{gzip}}(s) = \frac{|\text{gzip}(b(s))|}{|b(s)|}.$$

A larger value means lower compressibility under this particular algorithm. It is neither Shannon entropy nor a universal estimator of Kolmogorov complexity. The inverse convention $|b(s)|/|\text{gzip}(b(s))|$ has the opposite orientation and must not be substituted without changing all inequalities.

11 Experimental design

Medium contains one author. High interleaves excerpts from ten authors and genres. Thus the nominal factor “diversity” jointly changes composition, domain, marginal frequencies, and alphabet. The observed lengths are 1,000,000 and 999,998 characters. A two-character discrepancy is numerically negligible at this scale, but the stored experiment does not satisfy literal equality of sample size.

The historical checkpoints do not record their random seeds. The current pipeline fixes and stores seeds, but that change cannot reconstruct the random state of the four reported fits. A complete training run is the experimental unit. The 100 evaluation minibatches are repeated measurements of a fixed checkpoint, not 100 independent training replications.

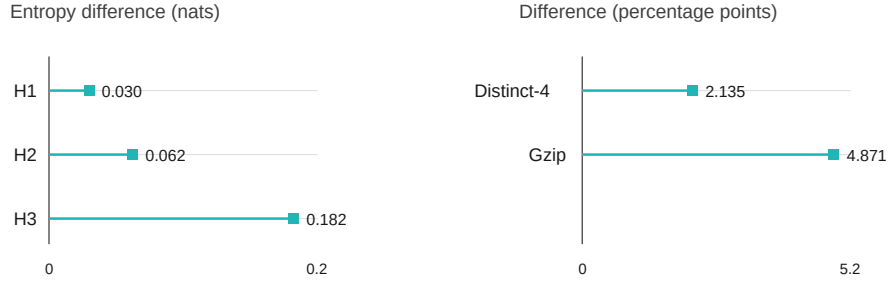
Table 1: Configuration of each historical fit.

Quantity	Value
Context length B	128
Minibatch size M	32
Residual dimension d	64
Blocks / attention heads	2 / 4
AdamW steps	5,000
Learning rate	3×10^{-4}
Independent repetitions	1

12 Results

Table 2: Descriptive corpus statistics. Entropies are in nats.

Corpus	N	$ \mathcal{V} $	H_3	D_4	ρ_{gzip}
Medium	1,000,000	94	7.4058	0.0378	0.3579
High	999,998	102	7.5880	0.0592	0.4066

**Figure 1:** High-minus-Medium differences. Entropy differences are measured in nats; D_4 and ρ_{gzip} differences are measured in percentage points. Separate axes prevent heterogeneous quantities from sharing a common bar scale.

The High-minus-Medium test differences are 0.1241 for the Transformer and 0.2372 for the LLaMA-style model. The corresponding gap differences are -0.0277 and -0.0251 . These signs describe the four stored checkpoints. There are no independent repetitions from which to estimate variation across initializations. Furthermore, High models have more parameters because $K = 102$ rather than $K = 94$.

The new evaluator produces L_{uni} , bpc, G_{ctx} , and G_{rel} . They are not inserted into the historical table until all four checkpoints have been evaluated under one versioned protocol. This restriction prevents a report from silently combining artifacts generated under different evaluation procedures.

13 Logical scope and identifiability

The observations establish only

$$\Delta_{\text{test}} > 0, \quad \Delta_{\text{gap}} < 0$$

Table 3: Empirical risks of the historical checkpoints. Parameter counts include vocabulary-dependent layers.

Model	Corpus	Train	Test	Gap	Parameters
Transformer	Medium	1.7679	1.8355	0.0676	120,030
Transformer	High	1.9198	1.9597	0.0398	121,062
LLaMA-style	Medium	1.3645	1.4638	0.0993	143,424
LLaMA-style	High	1.6268	1.7010	0.0742	144,448

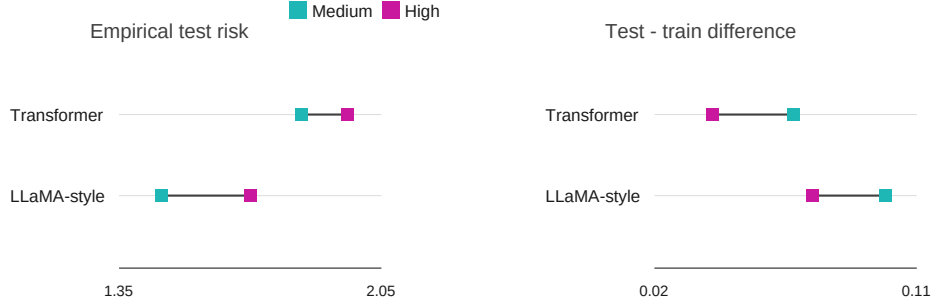


Figure 2: Test loss and the empirical test–train difference. Segments connect corpora within an architecture; they are neither trajectories nor confidence intervals. Perplexity is omitted because it is $\exp(\text{loss})$ and has exactly the same ordering.

for both architectures and for the stored realizations. They do not establish that diversity increases expected risk or acts as an implicit regularizer. A causal interpretation would require an intervention that changes diversity while holding domain, alphabet, length, and semantic distribution fixed, as well as independent training repetitions.

A smaller gap does not by itself order out-of-sample performance. A constant predictor may have a zero gap and high risk. Here the smaller High gap coexists with larger train and test losses. The phrase “less overfitting” can therefore refer only to an empirical difference of losses, not to superior prediction.

An identifiable follow-up design should include:

1. a family of corpora generated by an explicit random mechanism;
2. an exactly shared character budget and a vocabulary fitted on train;
3. matched parameter counts and target-position budgets;
4. multiple seeds treated as independent experimental units;
5. prespecified contrasts for raw loss, contextual gain, and interactions with architecture;
6. blockwise evaluation or inference valid for dependent sequences.

Varying data size and compute in addition would test whether diversity changes the scaling rate, which is closer to the effective-token question than a single-budget comparison.

14 Conclusion

The experiment separates three mathematical objects: descriptive diversity of an observed string, predictive risk of a conditional model, and the empirical test–train difference. In the

stored runs, High has larger diversity functionals, larger test loss, and a smaller gap. The evidence does not identify a causal relation among these facts. The reproducible contribution is a formal specification of the pipeline, an audit of its confounders, and an evaluation scheme that compares absolute risk with contextual gain over a unigram reference. The effective-token hypothesis remains open for a replicated and controlled design.

Tool disclosure

Language-model assistants were used for code review, manuscript restructuring, and algebraic checks. Numerical statements are derived from versioned repository artifacts. Responsibility for the content and conclusions remains with the author.

References

- [1] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020. URL <https://arxiv.org/abs/2001.08361>.
- [2] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022. URL <https://arxiv.org/abs/2203.15556>.
- [3] Ernie Chang, Matteo Paltenghi, Yang Li, Pin-Jie Lin, Changsheng Zhao, Patrick Huber, Zechun Liu, Rastislav Rabatin, Yangyang Shi, and Vikas Chandra. Scaling parameter-constrained language models with quality data. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 80–97, Miami, Florida, US, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-industry.8. URL <https://aclanthology.org/2024.emnlp-industry.8/>.
- [4] Philipp Petersen and Jakob Zech. Mathematical theory of deep learning, 2026. URL <https://arxiv.org/abs/2407.18384>.
- [5] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016. URL <https://arxiv.org/abs/1606.08415>.
- [6] Stefan Elfving, Eiji Uchibe, and Kenji Doya. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural Networks*, 107:3–11, 2018. doi: 10.1016/j.neunet.2017.12.012.
- [7] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986. doi: 10.1038/323533a0.
- [8] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL <https://papers.nips.cc/paper/7181-attention-is-all-you-need>.
- [9] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothee Lacroix, Baptiste Roziere, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023. URL <https://arxiv.org/abs/2302.13971>.
- [10] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021. URL <https://arxiv.org/abs/2104.09864>.

- [11] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015. URL <https://arxiv.org/abs/1412.6980>.
- [12] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL <https://arxiv.org/abs/1711.05101>.